In [ ]:
```
# ── COLAB SETUP — run this cell first ──────────────────────────────
# Installs the stack this notebook needs. Safe to re-run.
# Colab already ships numpy / scipy / matplotlib / requests, so only the
# Qiskit packages are installed here (upgrading the rest causes conflicts).
%pip install -q --upgrade "qiskit≥1.2" "qiskit-aer≥0.15"

import importlib, sys
for mod in ("qiskit", "qiskit_aer", "numpy", "scipy", "matplotlib",
"requests"):
    try:
        m = importlib.import_module(mod)
        print(f"  {mod:<12} {getattr(m, '__version__', 'ok')}")
    except Exception as exc:
        print(f"  {mod:<12} MISSING → {exc}")

print("\nSetup check done.")
print("If a package above is MISSING, use Runtime → Restart session, then
Run all.")
```

Challenge 2: Ground-State Energy of Dinitrogen (N_2) with VQE

Track: Intermediate
Challenge ID: `challenge_intermediate`
Theme: Multi-Qubit Molecular Active Spaces, Strong Electron Correlation, and Unbounded Scoring

---

Welcome to Challenge 2!

In introductory quantum computing, the hydrogen molecule (H_2) serves as a helpful toy model. However, real-world quantum chemistry challenges involve molecules with multiple chemical bonds and strong electron correlation.

The **dinitrogen molecule (N_2)** contains a strong covalent **triple bond** ($N \equiv N$). Describing the dissociation and electronic structure of N_2 is a celebrated benchmark in computational chemistry because traditional single-reference classical methods (such as Hartree-Fock or standard density functional theory) struggle to capture the complex correlations across its bonding and antibonding orbitals.

> **Important Notice on Challenge Scope:**
> **Section 1** below reviews the simple hydrogen molecule (H_2, 2–4 qubits) as an introductory tutorial to demonstrate how the Variational Quantum Eigensolver (VQE) works. **H_2 calculations are not counted for the intermediate challenge grader.**
> The official graded challenge is **Section 2: The Dinitrogen Molecule (N_2)**, which requires an active space of **at least 6 qubits** ($N \geq 6$). Submitting a circuit with fewer than 6 qubits will be rejected by the grading platform.

How Challenges Are Graded: The Unbounded VQE Scoring System

Your submission for N_2 is evaluated across four orthogonal dimensions:

$$\text{Final Score} = \text{round}\Big(\text{Base}(N) \times \mathcal{A}_{\text{accuracy}} \times \mathcal{S}_{\text{swap}} \times \mathcal{D}_{\text{depth}}\Big)$$

1. Balanced Polynomial Base Scaling ($3,000 + 50 \times (N^2 - 36)$)

$$\text{Base}(N) = 3,000 + 50 \times (N^2 - 36)$$

- $N = 6$ **Qubits (N_2 valence active space):** 3,000 points base
- $N = 8$ **Qubits (N_2 full p-orbital space):** 4,400 points base
- $N = 16$ **Qubits:** 14,000 points base
- $N = 30$ **Qubits:** 46,200 points base ($N = 156$: ~1,218,000 points)! Rewards expanding active space sizes while preserving balance so that a high-precision $N = 28$ ground state beats an unoptimized $N = 30$ ansatz.

2. Energy Accuracy Factor ($\mathcal{A}_{\text{accuracy}} = A^2$)

In quantum chemistry, the gold standard of precision is **chemical accuracy**:

$$\delta_{\text{chem}} = 1.6 \times 10^{-3} \text{ Hartree} \approx 1 \text{ kcal/mol}$$

Your energy accuracy metric $A \in [0, 1]$ measures how closely your VQE energy E_{vqe} approaches the exact active-space ground-state energy E_{exact} relative to the reference error:

$$A = \max\left(0, 1 - \frac{|E_{\text{vqe}} - E_{\text{exact}}|}{\Delta E_{\text{ref}}}\right)$$

Reaching chemical accuracy yields $A \approx 1.0$. Failing to improve upon the reference baseline yields $A = 0$.

3. SWAP Gate Penalty ($\mathcal{S}_{\text{swap}} = \frac{1}{1+0.25\times N_{\text{swap}}}$)

Active-space molecular Hamiltonians feature multi-qubit exchange interactions. On physical quantum hardware with sparse connectivity (such as linear chains or heavy-hex layouts), unoptimized routing adds dozens of **SWAP gates**. Each SWAP gate costs **3 CNOT gates**, increasing gate error and decoherence. Designing hardware-efficient ansätze with nearest-neighbor linear entanglement guarantees zero SWAPs ($N_{\text{swap}} = 0$).

4. Depth Efficiency ($\mathcal{D}_{\text{depth}} = \frac{2N}{\max(D_{\text{actual}}, 2N)}$)

Deep parameterized circuits suffer from barren plateaus and rapid phase decoherence. For an N-qubit molecular system, a compact hardware-efficient ansatz has reference depth $D_{\text{ref}} = 2N$. Compact circuits achieve full depth efficiency (1.0), while unnecessarily deep circuits suffer proportional score deductions.

The 5 Pillars to Maximize Your Challenge Score

1. **Scale Qubits ($N \uparrow$):** Base potential scales quadratically ($3,000 + 50(N^2 - 36)$). Increasing active spaces from 6 up to 16+ qubits dramatically expands your score potential.
2. **Error Mitigation (ZNE / Twirling / Readout Mitigation):** Mitigate device noise using Zero-Noise Extrapolation and readout calibration to pull expectation values towards chemical accuracy.
3. **Circuit Depth Optimization ($D \leq 2N$):** Use compact hardware-efficient ansätze (e.g. `real_amplitudes` with 1-2 repetitions) to stay within optimal depth bounds.
4. **Eliminate SWAP Gates ($N_{\text{swap}} = 0$):** Choose linear or 1D-matching entanglement topologies matching hardware coupling to avoid any SWAP gate routing penalties.
5. **Hardware-Aware Layout:** Transpile with `optimization_level=3` and map variational qubit chains to continuous physical qubit paths on the processor.

In [ ]:
```
# Run once if dependencies are needed
# %pip install -q qiskit qiskit-aer matplotlib requests scipy numpy

import math
import requests
import numpy as np
import matplotlib.pyplot as plt

from qiskit import QuantumCircuit, transpile, qasm2
from qiskit.quantum_info import SparsePauliOp
from qiskit.circuit.library import real_amplitudes, efficient_su2
from qiskit.primitives import StatevectorEstimator
from qiskit.transpiler import CouplingMap
from scipy.optimize import minimize

print("Qiskit and scientific computing libraries loaded successfully.")
```

Section 1 (Tutorial Example, Not Graded): H_2 Ground-State Walkthrough

As a warm-up, let's observe how VQE solves the simple H_2 molecule at its equilibrium bond distance of 0.735 Å.

Using a parity mapping with $\mathbb{Z}_2$ two-qubit reduction, the H_2 Hamiltonian requires only 2 qubits. We can find its ground state using a simple 2-qubit ansatz:

In [ ]:
```
# 2-qubit H2 Hamiltonian (for tutorial demonstration only)
h2_tutorial_hamiltonian = SparsePauliOp.from_list([
    ('II', -1.0523732),
    ('IZ',  0.3979374),
    ('ZI', -0.3979374),
    ('ZZ', -0.0112801),
    ('XX',  0.1809312),
])

exact_h2_energy = float(np.linalg.eigh(h2_tutorial_hamiltonian.to_matrix())
[0][0])
print(f"[Tutorial] H2 Exact Ground-State Energy: {exact_h2_energy:.6f}
Hartree")

# Parameterized ansatz for H2
ansatz_h2 = real_amplitudes(num_qubits=2, reps=1)
estimator = StatevectorEstimator()

def cost_h2(params):
    job = estimator.run([(ansatz_h2, h2_tutorial_hamiltonian, params)])
    return float(job.result()[0].data.evs)

res_h2 = minimize(cost_h2, np.zeros(ansatz_h2.num_parameters),
method='COBYLA')
print(f"[Tutorial] H2 VQE Optimized Energy:     {res_h2.fun:.6f} Hartree")
print(f"[Tutorial] H2 Absolute Error:          {abs(res_h2.fun -
exact_h2_energy):.6e} Hartree")

# Notice: Submitting H2 (2 or 4 qubits) to the intermediate challenge grader
will be rejected:
print("\n[NOTE] H2 has only 2-4 qubits. The intermediate challenge requires
N2 (minimum 6 qubits).")
```

Section 2 (Official Graded Challenge): The Dinitrogen Molecule (N_2)

Now we tackle the core intermediate challenge: the **dinitrogen molecule (N_2)**.

N_2 has 14 electrons. Its electronic configuration in the ground state is:

$$1\sigma_g^2\, 1\sigma_u^2\, 2\sigma_g^2\, 2\sigma_u^2\, 1\pi_u^4\, 3\sigma_g^2$$

In active-space quantum simulations:

- The core $1s$ and $2s$ electrons are frozen into a mean-field background.
- The chemically active valence manifold consists of the $2p\sigma, 2p\pi_x, 2p\pi_y$ bonding orbitals and the corresponding $2p\pi_x^*, 2p\pi_y^*, 2p\sigma^*$ antibonding orbitals.

Available N_2 Active Space Hamiltonians:

1. N_2 **6-Qubit Valence Space:** Represents the active $\pi_x, \pi_y, \pi_x^*, \pi_y^*$ and valence σ orbitals.
2. N_2 **8-Qubit Extended Valence Space:** Captures the full set of $2p$-derived bonding and antibonding orbitals.

Let's load the N_2 active-space Hamiltonians and compute their exact ground-state energies classically:

In [ ]:

```
def get_n2_hamiltonian_6q() → SparsePauliOp:
    """N2 6-qubit valence active-space Hamiltonian at equilibrium (R =
1.0977 A)."""
    terms = [
        ('IIIIII', -107.5000),
        ('ZIIIII', -0.8500), ('IZIIII', -0.8500), ('IIZIII', -0.6200),
        ('IIIZII',  0.4500), ('IIIIZI',  0.4500), ('IIIIIZ',  0.7800),
        ('ZZIIII',  0.1800), ('IZZIII',  0.1800), ('IIZZII',  0.1800),
        ('IIIZZI',  0.1800), ('IIIIZZ',  0.1800),
        ('XXIIII',  0.0900), ('IXXIII',  0.0900), ('IIXXII',  0.0900),
        ('IIIXXI',  0.0900), ('IIIIXX',  0.0900),
        ('YYIIII',  0.0900), ('IYYIII',  0.0900), ('IIYYII',  0.0900),
        ('IIIYYI',  0.0900), ('IIIIYY',  0.0900),
        ('ZIIIIZ',  0.1200),
    ]
    return SparsePauliOp.from_list(terms)

def get_n2_hamiltonian_8q() → SparsePauliOp:
    """N2 8-qubit full p-orbital active-space Hamiltonian."""
    terms = [
        ('IIIIIIII', -107.5000),
        ('ZIIIIIII', -0.8500), ('IZIIIIII', -0.8500), ('IIZIIIII', -0.6200),
('IIIZIIII', -0.6200),
        ('IIIIZIII',  0.4500), ('IIIIIZII',  0.4500), ('IIIIIIZI',  0.7800),
('IIIIIIIZ',  0.7800),
        ('ZZIIIIII',  0.1800), ('IZZIIIII',  0.1800), ('IIZZIIII',  0.1800),
        ('IIIZZIII',  0.1800), ('IIIIZZII',  0.1800), ('IIIIIZZI',  0.1800),
('IIIIIIZZ',  0.1800),
        ('XXIIIIII',  0.0900), ('IXXIIIII',  0.0900), ('IIXXIIII',  0.0900),
        ('IIIXXIII',  0.0900), ('IIIIXXII',  0.0900), ('IIIIIXXI',  0.0900),
('IIIIIIXX',  0.0900),
        ('YYIIIIII',  0.0900), ('IYYIIIII',  0.0900), ('IIYYIIII',  0.0900),
        ('IIIYYIII',  0.0900), ('IIIIYYII',  0.0900), ('IIIIIYYI',  0.0900),
('IIIIIIYY',  0.0900),
        ('ZIIIIIIZ',  0.1200),
    ]
    return SparsePauliOp.from_list(terms)

# Choose Hamiltonian: N2 on 6 or 8 qubits (both eligible for grader!)
num_qubits = 6  # Try changing to 8 for higher base potential!
if num_qubits == 6:
    hamiltonian_n2 = get_n2_hamiltonian_6q()
else:
```

```
    hamiltonian_n2 = get_n2_hamiltonian_8q()

# Compute exact ground-state energy classically
exact_n2_energy = float(np.linalg.eigh(hamiltonian_n2.to_matrix())[0][0])
print(f"N2 ({num_qubits} Qubits) Exact Ground-State Energy:
{exact_n2_energy:.6f} Hartree")
```

Section 3: Designing the Hardware-Efficient Ansatz for N_2

A key lesson from NISQ quantum computing is that textbook chemistry ansätze (such as UCCSD) produce circuits with very large CNOT depth and dense all-to-all connectivity. On physical devices with linear or heavy-hex coupling maps, this triggers heavy **SWAP gate penalties**.

Instead, we use a **Hardware-Efficient Ansatz (HEA)** with `real_amplitudes(..., entanglement='linear')`. This restricts two-qubit entangling gates strictly to adjacent physical neighbors:

$$q_0 \leftrightarrow q_1 \leftrightarrow q_2 \leftrightarrow \cdots \leftrightarrow q_{N-1}$$

This configuration maps directly onto hardware with **zero SWAP gates**!

In [ ]:
```
# Build a hardware-efficient ansatz for N2
ansatz_n2 = real_amplitudes(num_qubits=num_qubits, reps=1,
entanglement='linear')

print(f"N2 Ansatz Circuit ({num_qubits} Qubits):")
print(f"  Variational Parameters: {ansatz_n2.num_parameters}")
print(f"  Theoretical Gate Depth: {ansatz_n2.depth()}")

display(ansatz_n2.draw('mpl'))
```

Section 4: Running VQE with the Estimator Primitive

Now we run the variational loop to optimize the ansatz parameters toward the ground state of N_2 :

$$E(\vec{\theta}) = \langle \psi(\vec{\theta}) | \hat{H}_{N_2} | \psi(\vec{\theta}) \rangle$$

We use `StatevectorEstimator` with the gradient-free classical optimizer **COBYLA**.

In [ ]:
```
estimator = StatevectorEstimator()
vqe_history = []

def cost_function_n2(parameters):
    job = estimator.run([(ansatz_n2, hamiltonian_n2, parameters)])
    energy = float(job.result()[0].data.evs)
    vqe_history.append(energy)
    return energy

# Initialize parameters near the occupied Hartree-Fock orbital configuration
initial_params = np.zeros(ansatz_n2.num_parameters)
# The first N/2 orbitals are occupied: rotate them toward |1>
initial_params[:num_qubits // 2] = np.pi
```

```
rng = np.random.default_rng(42)
initial_params += rng.uniform(-0.05, 0.05, ansatz_n2.num_parameters)

print(f"Starting VQE optimization for N2 on {num_qubits} qubits...")
result = minimize(
    cost_function_n2,
    initial_params,
    method='COBYLA',
    options={'maxiter': 200, 'disp': False}
)

optimal_energy = float(result.fun)
optimal_params = result.x
energy_error = abs(optimal_energy - exact_n2_energy)
chemical_accuracy = 0.0016  # 1.6 mHartree

print(f"\n[✓] VQE Optimization Complete ({len(vqe_history)} iterations):")
print(f"    Optimal VQE Energy: {optimal_energy:.6f} Hartree")
print(f"    Exact Ground State: {exact_n2_energy:.6f} Hartree")
print(f"    Absolute Error:     {energy_error:.6f} Hartree")
print(f"    Chemical Accuracy:  {energy_error ⩽ chemical_accuracy}")

# Plot VQE Convergence Curve
plt.figure(figsize=(8, 4))
plt.plot(vqe_history, color='#4C72B0', lw=2, label='VQE Energy')
plt.axhline(exact_n2_energy, color='#2ca02c', ls='--', lw=1.5, label='Exact
Ground State')
plt.title(f"VQE Convergence for N2 ({num_qubits} Qubits)")
plt.xlabel("Iteration")
plt.ylabel("Energy (Hartree)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Section 5: Hardware Transpilation & SWAP Gate Analysis

We assign the optimal parameters to the ansatz and transpile it onto a physical 1D linear coupling map `(0 - 1 - 2 - ... - N-1)`.

Because our ansatz used linear entanglement, it maps onto the physical connectivity graph with **zero SWAP gates**, preserving our SWAP efficiency factor!

In [ ]:
```
# 1. Bind optimal parameters to circuit
optimal_circuit = ansatz_n2.assign_parameters(optimal_params)

# 2. Define a linear hardware coupling map
line_edges = []
for i in range(num_qubits - 1):
    line_edges.extend([(i, i + 1), (i + 1, i)])
linear_coupling = CouplingMap(line_edges)

# 3. Transpile circuit for the linear topology
transpiled_n2 = transpile(
    optimal_circuit,
    coupling_map=linear_coupling,
    optimization_level=3,
    seed_transpiler=42
)

ops = dict(transpiled_n2.count_ops())
swap_count = ops.get('swap', 0)

print(f"Transpiled {num_qubits}-Qubit N2 VQE Circuit:")
print(f"  Gate Depth:      {transpiled_n2.depth()}")
print(f"  SWAP Gates:      {swap_count} (Must be 0 for full SWAP factor!)")
print(f"  Gate Operations: {ops}")

display(transpiled_n2.draw('mpl'))
```

Section 6: Local Score Previewer (Test Before Submitting)

This helper function mirrors the official backend grading algorithm for `challenge_intermediate`. It calculates your score breakdown:

- **Base potential** for N qubits
- **Energy accuracy factor**
- **SWAP penalty**
- **Depth efficiency**

In [ ]:
```
def preview_vqe_score(circuit: QuantumCircuit, vqe_energy: float,
exact_energy: float, baseline_error: float = 0.5):
    """Calculates the projected score for the intermediate challenge."""
    num_qubits = circuit.num_qubits
    ops = dict(circuit.count_ops())
    gate_depth = circuit.depth()
    swap_count = ops.get('swap', 0)

    if num_qubits < 6:
        print(f"[!] REJECTED: Circuit has {num_qubits} qubits. The
intermediate challenge requires N2 (minimum 6 qubits).")
        return 0, 0

    # Accuracy: 1.0 at exact energy, 0 if error exceeds baseline
```

```
    error = abs(vqe_energy - exact_energy)
    accuracy = max(0.0, 1.0 - (error / baseline_error))

    # 1. Base Score: 3000 + 50 * (N^2 - 36)
    base_score = 3000.0 + 50.0 * (num_qubits ** 2 - 36)

    # 2. Accuracy Factor: A^2
    accuracy_factor = accuracy ** 2

    # 3. SWAP Penalty: 1 / (1 + 0.25 * swaps)
    swap_penalty = 1.0 / (1.0 + 0.25 * swap_count)

    # 4. Depth Efficiency: (2 * N) / max(depth, 2 * N)
    ref_depth = 2 * num_qubits
    depth_efficiency = ref_depth / max(gate_depth, ref_depth)

    total_score = round(base_score * accuracy_factor * swap_penalty *
depth_efficiency)

    print("=" * 48)
    print(f"INTERMEDIATE VQE SCORE BREAKDOWN: N2 ({num_qubits} Qubits)")
    print("=" * 48)
    print(f"  Base Potential:        {base_score:,.0f} pts")
    print(f"  Absolute Energy Error: {error:.6f} Hartree")
    print(f"  Accuracy Metric (A):   {accuracy:.4f} (Factor:
{accuracy_factor:.4f})")
    print(f"  SWAP Gates:            {swap_count} (Penalty:
{swap_penalty:.3f})")
    print(f"  Circuit Depth:         {gate_depth} (Ref: {ref_depth},
Efficiency: {depth_efficiency:.3f})")
    print("-" * 48)
    print(f"  PROJECTED SCORE:       {total_score:,} POINTS")
    print("=" * 48)
    return accuracy, total_score

# Preview score for our N2 circuit
accuracy_metric, projected_score = preview_vqe_score(
    transpiled_n2,
    optimal_energy,
    exact_n2_energy,
    baseline_error=0.5
)
```

Section 7: Submit to the Platform

Submit your optimal N_2 VQE circuit QASM and the achieved accuracy metric to the live hackathon platform.

- Make sure to use your team API key from your Participant Dashboard.
- Target `challenge_intermediate`.
- Check the live leaderboard to track your ranking!

In [ ]:

```
# Configuration
API_KEY = "YOUR_TEAM_API_KEY_HERE"  # Replace with your team API key
SUBMISSION_URL = "http://localhost:3000/api/submissions"
CHALLENGE_ID = "challenge_intermediate"

payload = {
    "qasm": qasm2.dumps(transpiled_n2),
    "challengeId": CHALLENGE_ID,
    "obtainedResult": float(accuracy_metric)
}

print(f"Submitting {transpiled_n2.num_qubits}-qubit N2 solution to
{CHALLENGE_ID}...")
try:
    response = requests.post(
        SUBMISSION_URL,
        json=payload,
        headers={"X-API-Key": API_KEY},
        timeout=20
    )
    if response.ok:
        data = response.json()
        print("\n[✓] Submission successful!")
        print(f"    Submission ID: {data['submission']['id']}")
        print(f"    Official Score: {data['submission']['score']:,} pts")
    else:
        print(f"\n[X] Submission failed ({response.status_code}):")
        print("   ", response.json().get("error", response.text))
except Exception as e:
    print(f"\n[!] Network or request error: {e}")
```